Let’s consider the request/response pattern commonly used in web applications to establish synchronous interactions over HTTP. A client sends an HTTP request to a server that performs some computation, and it then replies with an HTTP response.

In web applications running in a Servlet container like Tomcat, requests are processed based on a model called *thread-per-request*. For each request, the application dedicates a thread exclusively to handling that specific request; the thread will not be used for anything else until a response is returned to the client. When the request-handling involves intensive operations like I/O, the thread will block until the operations are completed. For example, if a database read is required, the thread will wait until data is returned from the database. That’s why we say that this type of processing is *synchronous* and *blocking*.

Tomcat is initialized with a thread pool that’s used to manage all incoming HTTP requests. New requests will be queued when all threads are in use, waiting for a thread to become free. In other words, the number of threads in Tomcat defines an upper limit to how many requests are supported concurrently. This is very useful to remember when debugging performance issues. If the thread concurrency limit is hit continuously, you can always tune the thread pool configuration to accept more workload. With traditional applications we would add more computational resources to a specific instance. For cloud native applications, we rely on horizontal scaling and deploying more replicas.

NOTE In some applications that have to respond to high demands, the thread-per-request model might not be ideal, because it doesn’t use the available computational resources in the most efficient way due to the blocking. In chapter 8, I’ll introduce an *asynchronous* and *non-blocking* alternative with Spring WebFlux and Project Reactor, adopting the reactive programming paradigm.

Spring MVC is the library included in the Spring Framework to implement web applications, either full MVC or REST-based. Either way, the